Derivatives: Contracts, Models, and Markets · Technical Note 39

Machine Learning in Derivatives

Wulin Suo

This note accompanies Derivatives: Contracts, Models, and Markets online rather than in print. It separates three activities that travel under the name machine learning in derivatives—approximating a pricing function the desk already knows how to compute, estimating a conditional expectation from simulated paths, and learning a hedging strategy where frictions make replication impossible—and asks of each whether it changes an answer or only the time taken to get one. It works through pricing surrogates and the domain over which they can be trusted, differential learning, deep hedging judged against a tuned no-trade band, neural regression for exercise decisions in high dimension, and what breaks and how such models are governed. Chapter 38 of the book defines the expected shortfall that deep hedging minimizes, and [TN38] *derives the representation that makes it trainable; Chapter 39 takes up the validation of learned models from the supervisor's side. The note keeps the questions it carried as a chapter. Its companion notebook reproduces every number.*

Three quite different activities travel under one name, and treating them as one subject is the main reason the literature is hard to read.

The first is approximating a function you already know how to compute. A pricing model is a map from parameters to prices; evaluating it is slow; a network can learn the map and evaluate it in microseconds. Nothing about the answer changes—only the time taken to get it.

The second is estimating a conditional expectation from simulated paths. That is Section 20.11's American Monte Carlo with a different basis. The regression was always there; replacing polynomials with a network is a change of basis, not of method.

The third is different in kind. It asks for a hedging strategy under transaction costs, discrete rebalancing, and position limits—a problem the classical theory cannot state, because replication does not exist once frictions are present and the theory has nothing to say about what to do instead. Here the machine is not accelerating an answer; it is producing an answer where there was none.

One test organizes the note: **does the method change an answer, or only the time taken to get one?** Both are worth having and they are not the same thing, and the difference decides how much validation the result needs. The note is written for a reader who will implement or validate a learned model; a reader who wants only the conclusions can take Section 14 on its own.

Sections 2 to 5 are the first task: pricing surrogates, the domain over which they can be trusted, the pricer's errors they inherit, and differential learning. Sections 6 to 10 are the third, deep hedging, judged against the right benchmark and read through its objective and its simulator, and Section 11 is the second, exercise decisions in high dimension. Sections 12 to 14 say what breaks, how such models are governed,

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

and where the methods genuinely help; Section 15 says who uses them and why, and Section 16 how the note's results were obtained.

1 THREE TASKS, ONE SET OF TOOLS

All three use the same object: a parametrized function f_θ, a scalar objective, and gradient descent on θ. What differs is where the training data comes from and what the objective means.

Supervised approximation. Labels come from a pricer you trust. The objective is fit. The best achievable outcome is to reproduce the pricer exactly, so the method inherits the pricer's ceiling—and, as Section 4 shows, its floor.

Regression on simulated paths. Labels are noisy realizations whose conditional mean is the quantity wanted. The objective is the same least squares Section 20.11 already used, and the network is one basis among many.

Stochastic control. There are no labels at all. The objective is a risk measure applied to a terminal profit and loss, and the network is a decision rule. This is the only one of the three that can produce something classical methods do not contain.

The architectures in this note are deliberately small—the largest network here has 1,345 parameters and the hedging policy has 1,217. Nothing in the results depends on depth or width, which is itself worth knowing: where these methods help in derivatives, they help for structural reasons rather than through capacity.

2 PRICING SURROGATES

A calibration routine evaluates a model thousands of times. If each evaluation costs half a millisecond, a single calibration costs seconds, and a desk recalibrating hundreds of surfaces intraday cannot afford it. The surrogate replaces the pricer inside the loop, not the model.

Learn implied volatility rather than price. Prices span orders of magnitude across strike and maturity, so a fitted network spends its capacity on the large ones; implied volatilities live in a narrow band and are the quantity the calibration objective is written in anyway.

Example 2.1 A network that prices Heston

Sample the Heston parameters and the contract uniformly over

$$v_0, \theta \in [0.01, 0.16], \quad \kappa \in [0.5, 3], \quad \sigma_v \in [0.1, 1.0], \quad \rho \in [-0.9, 0],$$

with $K/S \in [0.7, 1.3]$ and $T \in [0.1, 2]$, price by quadrature, and invert to implied volatility. Points whose option carries less than 0.1% of spot in time value are discarded, because an implied volatility is not defined in any useful sense for a quote that does not exist. That leaves 23,310 training points, and 17 seconds of quadrature to generate them.

A network of **1,345 parameters**—two hidden layers of 32 units—trains in 31 seconds and reproduces the map on 4,867 unseen points, measured by root mean

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

squared error (RMSE):

RMSE	**15.1 bp** of volatility
Mean absolute error	11.1 bp
95th percentile	29.8 bp
Worst	123 bp

Fifteen basis points is comfortably inside a bid–ask spread, so for calibration the surrogate is indistinguishable from the pricer. The speed is the point:

	Quadrature	**Surrogate**
Per price	0.501 ms	0.00255 ms
One calibration (200 quotes, 60 evaluations)	6.0 s	**0.031 s**

A factor of 196. That is the whole of the contribution, and it is a large contribution: it is the difference between recalibrating a book overnight and recalibrating it on every quote request. It is also entirely a statement about computation and not at all about finance—the prices are the same prices.

This is why Chapter 21 could describe rough volatility as having moved into production. Nothing about the model changed; what changed is that calibrating it stopped costing minutes. A model becomes usable when its calibration becomes affordable, and that is a sentence about hardware and numerical method rather than about the model's realism.

3 THE TRAINING SET IS THE DOMAIN OF VALIDITY

A surrogate is an interpolation, and interpolations do not extrapolate. The danger is that the failure is silent: the network returns a number of the usual size, with the usual number of digits, and nothing in the output indicates that the inputs were outside the region it was fitted on.

Example 3.1 Outside the training box

Take the network of Example 2.1, unchanged, and evaluate it at parameters beyond the training range:

	RMSE	**Worst**
Inside the training box	15.1 bp	123 bp
Vol-of-vol raised to 1.6	66.5 bp	371 bp
Maturity extended to 5 years	406.6 bp	1068 bp
Strike extended to 1.9× spot	346.7 bp	1141 bp

Fifteen basis points of error become four hundred—a factor of 27—and a ten-volatility-point error is not a calibration nuisance—it is a different market. The degradation is worst in maturity, which is the direction a desk is most likely to step outside the box without noticing: a five-year quote looks like the same

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

kind of object as a two-year one.

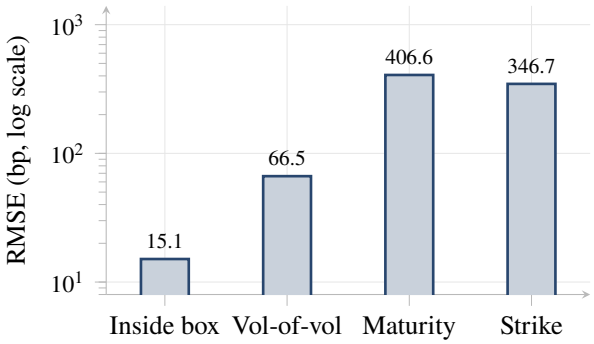


Figure 1: Surrogate RMSE inside the training box and at three extrapolations. Log scale.

Figure 1 needs a log scale, which is the point. Inside the box the error is a rounding difference; one step outside it—on a maturity the network was never shown—it exceeds the bid–ask on the instrument being priced. The network gives no sign that it has left the region it was fitted on, which is why the boundary has to be enforced from outside.

The remedy is trivial and the discipline is not. A surrogate should refuse to answer outside its training range rather than extrapolate, which costs one bounds check per evaluation. What makes this hard is organizational: the network is fast precisely so that it can be called from places that do not know what it is, and the bounds check has to travel with it.

There is a genuine virtue here that classical methods lack. A closed-form approximation has a region of validity that is a matter of judgment and is usually undocumented; a surrogate's region of validity is a box someone wrote down. The boundary is explicit, which makes it auditable in a way that "this expansion is good for small $\sigma\sqrt{T}$" is not.

The point deserves a formal statement, because it is about what a model *is*. A trained network is a function $f_{\hat{\theta}}$ whose parameters minimize an empirical objective over a sample drawn from a distribution supported on a set $\mathcal{X}$. Nothing in the construction constrains $f_{\hat{\theta}}$ off $\mathcal{X}$: the objective is blind there, the gradients that shaped $\hat{\theta}$ never visited it, and the behavior outside is whatever the network's smoothness happens to produce.

So $\mathcal{X}$ is part of the model in the same sense the parameters are. Two networks with identical architectures, identical weights, and different documented boxes are *different models*, because one will refuse a query the other answers. Documenting the architecture and omitting the training set documents the least important part.

4 A SURROGATE LEARNS ITS PRICER, BUGS INCLUDED

The most instructive failure in preparing this note was not a modeling failure.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

Case Study 4.1 The network that learned a bug

The first surrogate trained for Example 2.1 reported a worst-case error of **1,814 basis points**—eighteen volatility points—on a test point inside its own training box. The mean error was a respectable 15.8 bp, so the fault was invisible in aggregate.

Sorting the test set by error and inspecting the worst points showed what they had in common: all were deep out of the money, short dated, and at high vol-of-vol. Pricing them directly revealed the cause. The quadrature was returning negative call prices—-0.0023, -0.0090, -0.000033—for options whose true value was of order 10^{-4} to 10^{-6}. The Gauss–Legendre rule with 200 nodes had not converged where the characteristic function oscillates fastest.

The labels were wrong, and the network had learned them and smoothed them. Raising the quadrature to 600 nodes converged the pricer to 6×10^{-9}, and the retrained network's worst error fell from 1,814 bp to **123 bp**.

Two lessons, and the second is the uncomfortable one. A surrogate cannot be more accurate than its labels, which is obvious. But a surrogate is smooth, and smoothing a bug makes it harder to find than it was in the original. The quadrature announced its failure by returning a negative price—an impossible value that any check would catch. The network replaced that with a plausible volatility of the right order of magnitude. The bug survived and stopped being detectable.

The practical rule follows directly. Validate the *labels* before validating the fit, and validate them where they are hardest to compute rather than where they are easy. A training set assembled by a pricer running at default settings across a wide parameter box will contain bad labels, because default settings are chosen for the middle of the box.

5 DIFFERENTIAL LEARNING

When labels come from simulation they are noisy, and the noise sets the sample size. But a simulated path yields more than a payoff: it yields the payoff's derivative with respect to the inputs, at essentially no additional cost—Section 20.10's pathwise differentiation, computed for a whole book by Chapter 32's adjoint algorithmic differentiation.

Derivative labels carry shape information that value labels do not. A single noisy payoff constrains the fitted function at one point; a payoff together with its slope constrains the function and its tangent, which is worth several nearby points.

Example 5.1 The same paths, twice as informative

Learn the price of a one-year call struck at 100 as a function of spot, from single-path payoffs simulated at random spots—the setting in which the label is pure noise around the right answer. Each path also produces its pathwise delta, $\mathbf{1}\{S_T > K\}\, S_T/S_0$, from the same draw.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

Using the derivative labels by Taylor augmentation—each path contributes its own point plus two neighbors implied by its slope—and holding the *number of simulated paths* fixed:

Paths	Plain RMSE	With derivatives	Improvement
256	7.6439	2.5109	3.04×
1,024	2.0616	1.1881	1.74×
4,096	0.9150	0.3891	2.35×
16,384	0.3421	0.2394	1.43×

The improvement is consistent in direction and noisy in size, averaging about a factor of two in RMSE. Read across the table instead of down it and the economic statement is clearer: the derivative-trained fit at 4,096 paths is about as accurate as the plain fit at 16,384. Derivative labels buy roughly a factor of three to four in simulation budget, for information the simulation had already computed and was throwing away. How the labels are used matters more than it looks—a mistake made in preparing this note, caught only by removing the derivative labels and watching the fit fail to degrade.

The construction is worth stating, because the obvious alternative is easy to get wrong. Let the target be $V(x) = \mathbb{E}[Y \mid X = x]$, estimated from pairs (x_i, y_i) with y_i a single noisy draw; pathwise differentiation supplies y'_i with $\mathbb{E}[y'_i \mid X = x_i] = V'(x_i)$ at no extra simulation cost. The cheapest correct way to use it is a first-order expansion—each path contributing not one constraint but a short segment,

$$(x_i,\ y_i), \qquad (x_i \pm h,\ y_i \pm h\, y'_i)$$

fitted by ordinary least squares. This adds data rather than a penalty term, so no second-order differentiation of the network is required.

The alternative—penalizing $\|f'_\theta(x_i) - y'_i\|^2$ directly—is the form usually written down. Its gradient requires differentiating the network's *Jacobian* with respect to the parameters, a second-order object; substituting a finite-difference estimate and back-propagating through the value output instead yields a smoothness penalty that improves fits for reasons unrelated to the labels. The test is to replace the derivative labels with noise: a correct implementation degrades sharply and a smoothness penalty does not.

The reason this matters beyond the sample size is Chapter 32's constraint. A model that cannot produce its risk overnight will not be used, and adjoint differentiation is what made those sensitivities affordable. Differential learning is the observation that the same derivatives, once computed, are also the best training data available.

6 DEEP HEDGING

Everything so far accelerates a calculation. *This section changes an answer.*

Chapter 13 defined an option's price as the cost of replicating it, and Chapter 18 hedged one in practice: rebalance to the Black–Scholes delta, accept the error

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

from rebalancing discretely, and Section 18.9 measured what that error costs. The construction assumes trading is free. Introduce a proportional transaction cost and replication becomes impossible—not expensive, impossible—because the replicating strategy has unbounded variation and would incur infinite cost.

The classical response is to hedge approximately and argue about how. Rebalance on a schedule, or when the delta has moved by more than a band, and choose the frequency or the band width to trade off hedging error against cost. Both keep the classical goal, which is to cancel the delta term of Chapter 18's expansion, equation (18.1), as nearly as cost allows. The deep hedging response is to state the problem the classical theory declines to state: drop that goal and choose the whole strategy at once to minimize a risk measure of the terminal profit and loss.

$$\min_{\theta} \ \rho\left(-\Pi + \sum_{t} \delta_t^{\theta} \left(S_{t+1} - S_t\right) \ - \ c \sum_{t} S_t \left|\delta_t^{\theta} - \delta_{t-1}^{\theta}\right| \ - \ \text{payoff} \right) \qquad \text{(TN39.1)}$$

where δ^{θ} is a decision rule with parameters θ, Π the premium received, c the cost rate, and ρ a risk measure.

Written directly, an expected shortfall averages over a tail whose boundary moves with the parameters, which is awkward to differentiate. The Rockafellar–Uryasev representation removes the difficulty, written here with the conditional value at risk (CVaR), which is Chapter 38's expected shortfall under the name the optimization literature uses:

$$\text{CVaR}_{\alpha}(L) \ = \ \min_{w \in \mathbb{R}} \left\{ w + \frac{1}{1-\alpha} \, \mathbb{E}\left[(L - w)^{+} \right] \right\} \qquad \text{(TN39.2)}$$

with the minimizing w the value at risk ([TN38] derives both), carried as one extra scalar trained alongside θ. The objective becomes an ordinary expectation of a piecewise-linear function, and its gradient is $\mathbf{1}\{L > w\}/(1-\alpha)$: only paths in the tail contribute. At $\alpha = 0.95$ and a batch of 2,048, about a hundred paths carry the entire signal—which is why this objective needs larger batches than a least-squares fit and why its training curve is noisier. The kink at $L = w$ costs nothing, by the argument that licenses pathwise differentiation of a call payoff at the strike.

The policy's own output is an input at the next date, and that is what makes the gradient interesting. Since trading *into* tomorrow's holding is charged on the distance from today's,

$$\frac{\partial \Pi}{\partial \delta_t} = \left(S_{t+1} - S_t\right) - c\, S_t \, \text{sgn}\left(\delta_t - \delta_{t-1}\right) + c\, S_{t+1} \, \text{sgn}\left(\delta_{t+1} - \delta_t\right)$$

so a cost-aware policy cannot be myopic—today's holding changes tomorrow's fee, which is the formal reason the current position must be an input. The total derivative then satisfies a backward recursion over the schedule, accumulating $\partial L / \partial \theta$ at each date and giving the exact gradient in one sweep.

Two consequences follow that are economic rather than architectural. The recursion multiplies Jacobian factors, so a policy whose sensitivity to its own previous holding

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

exceeds one in magnitude produces gradients growing geometrically backward—the exploding-gradient problem, arriving through the economics. And the whole path must be held in memory, since every date's activations are needed on the way back, so memory grows linearly in the number of rebalancing dates. That is what limits the schedules that can be trained, not the size of the network.

Two things about (TN39.1) deserve emphasis before any numbers. The risk measure is part of the specification and not a detail, as Section 9 shows. And the strategy is a function of state, so it must be given the state: here the moneyness, the time remaining, and the position currently held—without which no cost-aware rule can be expressed at all, since the cost of trading depends on where you already are.

7 WHAT THE NETWORK LEARNS

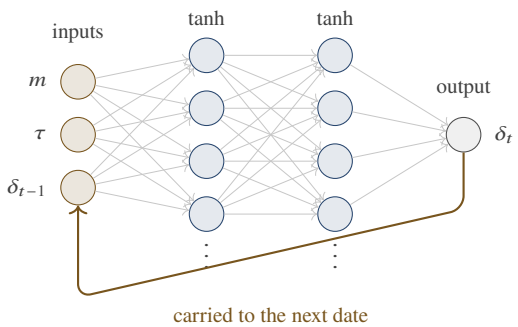


Figure 2: The hedging policy: two hidden layers, with the chosen holding carried back as an input. Schematic; the layers hold 32 units each.

Figure 2 is the whole architecture. Two hidden layers, 1,217 parameters, and one feature that is not decoration: the holding chosen at one date re-enters as an input at the next, which is what lets the policy weigh a trade against the fee it will pay to undo it.

Example 7.1 A short call, hedged three ways

Sell one at-the-money call: $S_0 = K = 100$, one month, 20% volatility, a zero interest rate ($r = 0$, here and in every hedging example of this note), 30 daily rebalancing dates, a Black–Scholes premium of 2.3030. Hedge it by three rules on the same 200,000 paths.

Black–Scholes delta rebalances to the model delta every day. *The band* rebalances to the model delta only when the holding has drifted more than w away from it, with w chosen by grid search to minimize the same objective the network optimizes—this is the benchmark that matters. *The learned policy* is a network of **1,217 parameters** trained on (TN39.1) with ρ the 95% expected shortfall.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

Cost	Strategy	Mean	Std dev	CVaR_{95}	Fees
0 bp	Black–Scholes delta	−0.0008	0.3600	0.8370	—
	learned	−0.0002	0.4097	**0.7522**	—
10 bp	Black–Scholes delta	−0.2741	0.3786	1.1964	0.2734
	band, $w = 0.03$	−0.2630	0.3838	1.1915	0.2622
	learned	−0.2448	0.4291	**1.0220**	0.2445
50 bp	Black–Scholes delta	−1.3675	0.5651	2.7361	1.3668
	band, $w = 0.13$	−1.0039	0.6473	2.5089	1.0032
	learned	−1.0301	0.5315	**1.9852**	1.0299

Read the 50 bp block carefully. The tuned band spends 1.0032 and the learned policy spends 1.0299, only 2.7% more, and their mean outcomes are within 3% of each other. But the expected shortfall is 2.5089 against 1.9852, a 21% improvement at almost the same cost.

So the network's advantage is not that it trades less. It is where it trades.

Two calibration checks confirm the arithmetic. At zero cost every strategy should be close to a fair game, and both have a mean within 0.002 of zero against a premium of 2.303; the band is not listed there because without costs its optimal width is zero and it is daily delta. And the optimal band widens as costs rise—0.03 at 10 bp, 0.13 at 50 bp—which is what the classical theory of no-trade regions requires.

What the policy actually does is visible in one picture, and it is not what a reader expects from a method described as opaque.

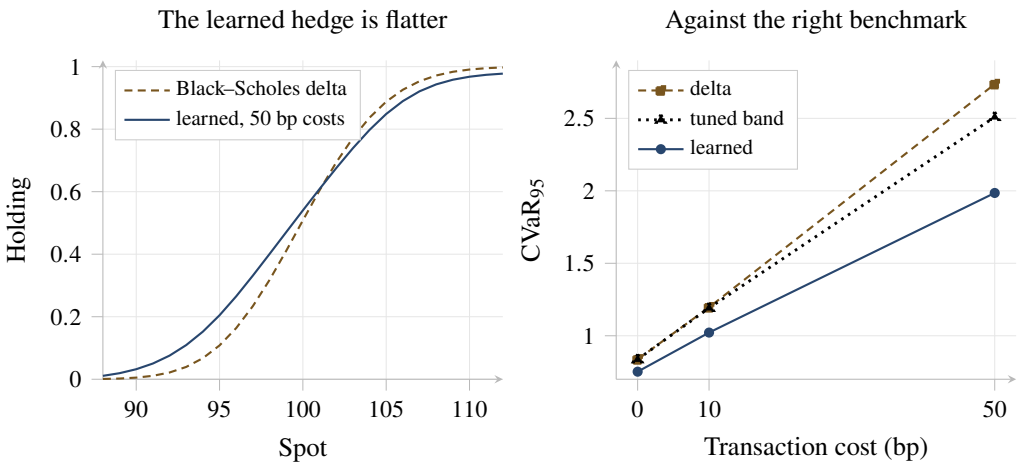


Figure 3: Left: holdings half a month before expiration, from a hedged starting position. Right: expected shortfall against transaction costs.

The left panel of Figure 3 shows a policy that is systematically flatter than the

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

model delta. Its slope near the money is 0.0690 against Black–Scholes–Merton's 0.0967—29% less gamma to rebalance against—and it holds 0.0109 shares where the model would hold 0.0009 and 0.9776 where the model would hold 0.9974. It never goes fully flat and never goes fully hedged, because both extremes are positions it would have to trade out of.

This is a recognizable object. The network has rediscovered the shape that the no-trade-region literature derives analytically: hedge less than the model says, and hedge less the more it costs. That is reassuring rather than disappointing. A method that produced something unrecognizable here would be harder to trust, not easier.

8 AGAINST THE RIGHT BENCHMARK

The comparison a marketing deck makes is against naive daily delta. On that basis the numbers are spectacular: at 50 bp the learned policy improves expected shortfall by **27.4%** and cuts fees by 24.6%.

The comparison that means something is against a tuned classical rule, and it tells a different and more useful story:

At 50 bp, learned versus. . .	**Naive delta**	**Tuned band**
Fees	−24.6%	+2.7%
CVaR_{95}	−27.4%	−20.9%

The entire cost saving is available from a no-trade band, an idea older than the machinery in this note and implementable in four lines. What the band cannot do is shape the tail, and that is what survives a like-for-like comparison.

The general form of this warning applies well beyond hedging. A learned method should be measured against the best classical alternative someone actually tuned, not against the first thing that came to hand—and the gap between those two benchmarks is usually larger than the gap the paper reports.

9 THE OBJECTIVE IS THE SPECIFICATION

The zero-cost row of Example 7.1 is the most important line in this note, and it looks at first like a failure.

With no transaction costs at all, the learned policy achieves an expected shortfall of 0.7522 against Black–Scholes delta's 0.8370—*10.1% better*—while its standard deviation is 0.4097 against 0.3600, *13.8% worse*.

Both numbers are correct and neither is a bug. The network was trained to minimize the 95% expected shortfall and it minimized the 95% expected shortfall. Variance was not in the objective, so variance was spent to buy tail improvement. A learned strategy is better at exactly what you asked for and worse at what you did not ask for, and the trade is invisible unless you look.

The consequence for a risk report is direct. If the desk optimizes expected shortfall and the risk system reports standard deviation, the method will appear to be failing at the moment it is working. Chapter 39's model governance question is usually posed

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

as whether the model is right; here the sharper question is whether the objective the model was given is the objective the institution actually has.

And the objective is a choice with no neutral option. Expected shortfall, variance, and an exponential utility give three different hedges for the same book. Classical delta hedging concealed this choice by having no objective at all—it replicates, and replication is unique when it exists. Once it does not exist, something has to be chosen, and deep hedging did not introduce the arbitrariness but made it explicit.

10 THE SIMULATOR IS AN ASSUMPTION

Deep hedging has no data. A surrogate is trained against a pricer that can be checked, and a regression is trained on payoffs that are realizations of the thing being estimated. (TN39.1) is trained on *simulated* paths, so whatever process generates those paths is an assumption the strategy inherits—and it is the one assumption the method cannot test, because the method never sees anything else.

The classical rules carry the same assumption and carry it more lightly. Black–Scholes delta is derived under geometric Brownian motion too. The difference is one of degree: a formula uses the model, and an optimizer exploits it.

Example 10.1 The same policy, a different world

Take the policy trained in Example 7.1 at 50 bp, change nothing about it, and evaluate it on paths from a Merton jump diffusion—matched to the same 20% total volatility, so the only difference is the shape of the daily moves. Jumps arrive 0.5 times a year with a mean of −5% and a volatility of 5%, and the diffusion volatility falls to 19.4% to hold the total at 20%. The kurtosis of daily returns rises from 3.00 to 10.09.

	As trained (GBM)		With jumps	
Strategy	**CVaR$_{95}$**	**Fees**	**CVaR$_{95}$**	**Fees**
Black–Scholes delta	2.7361	1.3668	3.1526	1.3555
band, $w = 0.13$	2.5089	1.0032	2.9034	0.9934
learned	**1.9852**	1.0299	**2.5957**	1.0222
Learned versus the band	−20.9%		−10.6%	

Half of the advantage disappears. The learned policy still wins, but the margin falls from 20.9% to 10.6% on a change that left total volatility exactly where it was.

The sharper way to read the table is by how much each strategy degrades: delta by 15.2%, the band by 15.7%, and the learned policy by **30.8%**. The strategy that was optimized hardest for the training distribution is the one that suffers most when the distribution is wrong, and it is roughly twice as sensitive as the rules that were merely derived under it.

This is not an argument against the method. It is an argument about what has to be delivered alongside it: a deep hedging result is a statement about a

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

simulator, and a desk that reports the strategy without reporting the process it was trained on has disclosed the less important half.

The practical response is to train on the uncertainty rather than on a point. Simulating from a mixture of plausible dynamics, or from bootstrapped historical returns, produces a policy that is worse on any single model and less sensitive across the set—which is the same trade Section 9 identified, one level up. What cannot be done is to skip the choice.

11 EXERCISE DECISIONS AND DIMENSION

Section 20.11 estimated a continuation value by regressing on a polynomial basis. Replacing the polynomials with a network changes the basis and nothing else: the bias is still downward, for Section 20.11's reason—a suboptimal exercise rule earns less—and the estimate is still a lower bound.

In low dimension this buys nothing, and the margin is not close.

Example 11.1 Where the polynomial basis actually gives out

A Bermudan put, $S_0 = K = 100$, $r = 5\%$, $\sigma = 20\%$, one year, twenty exercise dates, against a 6,000-step binomial reference of **6.0613**:

Basis	Value	Error	Time
Binomial tree	6.0613	—	0.28 s
Polynomial, degree 3	6.0485	−0.0128	0.1 s
Network, 2×16 units	5.7468	−0.3145	174 s

The network is 25 times further from the answer and 600 times slower than the tree.

The interesting case is five dimensions, and it did not behave as expected. A worst-of put on five underlyings with pairwise correlation 0.30, 100,000 paths:

Basis	Terms or parameters	Value	Time
Polynomial, degree 3	56 terms	14.7999	1.8 s
Polynomial, degree 4	126 terms	**14.8949**	4.7 s
Network, 2×16, 180 steps	385 parameters	13.5619	371 s
Network, 2×32, 600 steps	1,281 parameters	14.5327	2,237 s

Every value is a lower bound, so higher is better—a better basis finds a better exercise rule and earns more.

The polynomial wins, and the usual justification for the network turns out to be false. A degree-3 basis in five variables is $\binom{8}{3} = 56$ terms and a degree-4 basis is 126; against 100,000 paths neither is anywhere near a sample-size or memory limit. The claim that a polynomial basis becomes infeasible at five dimensions is simply wrong.

Nor is the network simply undertrained. Training it harder does not improve

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

the answer monotonically:

Network, 2×32	Value	Time
600 steps	**14.5327**	2,237 s
1,500 steps	13.7205	7,596 s

Identical paths, identical initialization, more than twice the training—and a materially worse answer.

The reason is not overfitting. Isolating a single exercise date with 43,320 paths held out, the network's out-of-sample error falls essentially monotonically and its train-test gap stays near zero throughout: 1,281 parameters against forty thousand points do not overfit.

What the held-out test does show is that the polynomial is the better estimator, not merely the cheaper one:

Basis	Held-out MSE	Train–test gap
Polynomial, degree 2 (21 terms)	75.74	−0.19
Polynomial, degree 3 (56 terms)	73.26	−0.15
Polynomial, degree 4 (126 terms)	**73.00**	+0.17
Polynomial, degree 5 (252 terms)	73.26	+0.77
Network, 2×32, best over 2,000 steps	74.14	−0.12

The network never reaches the polynomial's accuracy at any training length tried, and the polynomial has a visible optimum—degree 4 is best, degree 5 begins to overfit as its positive gap shows. Degree 4 also produced the best exercise rule in the table above, so the two diagnostics agree.

So the instability comes from the exercise rule rather than the fit. A continuation value is compared against an immediate payoff, and the comparison is a threshold: a small wobble in the fitted surface flips the decision on many paths at once, and each flip costs the lower bound. A regression that is slightly worse can produce an exercise rule that is much worse, which is why the value moves around while the held-out error barely does.

That is the transferable point, and it is not about networks. When a fitted function feeds a discrete decision, accuracy of the fit is the wrong thing to monitor—the quantity that matters is the stability of the decision boundary, and nothing in a training curve displays it.

So where does the network belong? Not at five dimensions, on this evidence. The crossover is somewhere higher than the textbook examples suggest, and anyone proposing a network basis should be asked for the polynomial comparison at the same path budget—Section 8's rule applied to the middle task rather than to hedging.

The general lesson repeats one this note has already drawn twice. The surrogate was worth having because it was measured against the quadrature it replaced; deep hedging was worth having because it was measured against a *tuned* band. Here the

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

same discipline reverses the expected conclusion, and it would have been easy not to run the comparison at all.

12 WHAT BREAKS

No-arbitrage is not enforced by construction—nothing in the loss function knows about butterflies—and the obvious inference from that is wrong, which Example 12.1 is included to establish.

Example 12.1 The surface that should have admitted an arbitrage and did not

Read one Heston surface off the surrogate of Example 2.1—$v_0 = 0.040$, $\kappa = 1.50$, $\theta = 0.045$, $\sigma_v = 0.60$, $\rho = -0.70$—and test it with the butterfly condition of Section 19.5, exactly as Example 19.5 tested market quotes.

The pointwise fit is ordinary: 17.7 bp RMSE across strikes, worst 42.3 bp. A butterfly is a second difference, so it amplifies error, and on a strike grid of spacing h the butterfly itself shrinks like h^2 while the fit error does not shrink at all. Refining the grid should therefore break it:

Strike spacing	Exact butterfly	Surrogate butterfly	Negative
2.0% of spot	$+2.257 \times 10^{-2}$	$+2.177 \times 10^{-2}$	none
0.5%	$+9.552 \times 10^{-4}$	$+8.737 \times 10^{-4}$	none
0.1%	$+3.456 \times 10^{-5}$	$+3.121 \times 10^{-5}$	none
0.05%	$+8.533 \times 10^{-6}$	$+7.694 \times 10^{-6}$	none

It does not break. Pushed further—to a five-year maturity, where the pointwise error reaches **698 bp**—the surface is *still* arbitrage-free.

A network is a smooth function of its inputs, so its error at neighboring strikes is almost the same error, and a second difference cancels whatever is smooth. The fit is wrong by 42 bp and wrong in the same direction at 0.995, 1.000, and 1.005 of spot, so the butterfly barely notices.

So the smoothness that made the pricer's bug undetectable in Section 4 is the smoothness that keeps this surface arbitrage-free.

The protection is narrower than it looks: smoothness protects the no-arbitrage conditions that are differences in the inputs the network was given. Strike is an input here, so butterflies are safe. A surrogate fitted separately for each maturity has no smoothness in maturity at all, and can violate the calendar condition of Section 19.5 while every individual smile is convex.

Extrapolation is silent, as Section 3 showed, and the failure mode is not a large error but a plausible one.

Results are not reproducible in the ordinary sense, and Example 12.2 shows that the interesting part is *which* results fail to reproduce.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

Example 12.2 Same code, same data, three strategies

Train the policy of Example 7.1 three times at 50 bp, changing only the random seed:

			Holding at spot		
Seed	**CVaR$_{95}$**	**Fees**	**95**	**100**	**105**
7	1.9852	1.0299	0.2047	0.5401	0.8485
17	2.0013	1.0631	0.2007	0.5478	0.8641
27	2.0003	1.0436	0.2062	0.5441	0.8652
Black–Scholes delta			0.1082	0.5081	0.8879

The performance reproduces and the strategy does not. Expected shortfall spans 1.9852 to 2.0013—**0.8%** of its mean—so all three runs are, as far as the objective is concerned, the same answer. But at a spot of 105 the three policies hold 0.8485, 0.8641, and 0.8652: a spread of 1.7 delta points on positions that are supposed to be identical. On a $100 million book that is a $1.7 million difference in stock held, between two desks running the same code on the same data.

What is reproducible is the shape. All three sit well above the Black–Scholes delta of 0.1082 at 95 and below 0.8879 at 105—every run learns to flatten, and they differ on how much.

This distinguishes the two uses cleanly. For a surrogate the deliverable is a price, and a price that moves 0.8% between runs would be unacceptable but a price that agrees to the fit error is fine. For a hedging policy the strategy is the deliverable, so seed dependence is not a numerical footnote—it is a difference in what the firm owns.

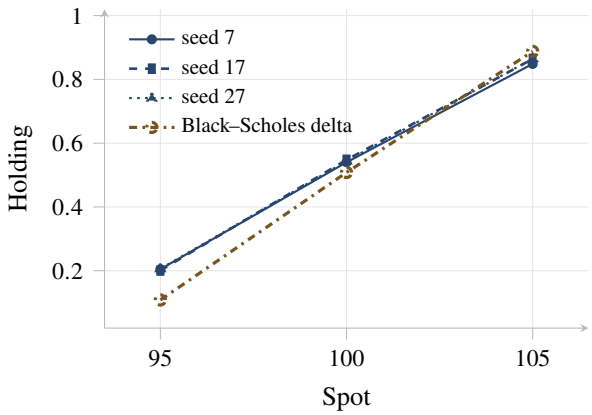


Figure 4: Three runs differing only in the random seed, against the Black–Scholes delta.

Figure 4 separates what reproduces from what does not. The three runs disagree on how much to hold, by up to 1.7 delta points, and agree on where they sit relative to the

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

delta—above it below the strike, below it above. The shape is the model's finding; the holding is an artifact of the seed, and only one of the two can be put in a risk report.

The practical answer is to fix seeds, version the weights as data rather than as code, and treat retraining as a model change requiring approval—which is a governance answer to a technical problem, and the only one available.

And the validation problem has no clean solution. A classical model is validated by checking its assumptions and its limiting cases. A network has no assumptions to check, so validation reduces to out-of-sample testing—which tests the region you sampled, and the failures in Section 3 were exactly in the region no one sampled.

13 GOVERNANCE

Chapter 39 treats model risk in general; three points here are specific to learned models.

The training data are part of the model. A surrogate's parameter box, a hedging policy's simulated dynamics, and the pricer that produced the labels are all model inputs in the sense that changing any of them changes the output. Documenting the architecture and omitting the training set documents the least important part.

Retraining is a model change. A network refitted on new data is a different model producing different numbers, and the natural cadence of retraining—often, quietly, automatically—collides with the approval cycle governance frameworks assume.

And the failure modes do not resemble the ones the frameworks were written for. A misspecified classical model is wrong in a direction you can reason about; a network trained on bad labels is wrong in a shape, as Section 4's smoothed bug illustrates. The supervisory question "is the model conservative?" has no answer for an object with no parameters that mean anything.

14 WHERE IT GENUINELY HELPS

Applying the note's test:

Surrogates change no answer and change the cost of getting one by two orders of magnitude, which makes previously unaffordable models usable. *Validation burden: low*, because the reference answer exists and can be checked.

Differential learning changes no answer and reduces the simulation budget by a factor of three or four, using derivatives the calculation already produced. *Validation burden: low*, for the same reason.

Neural regression for exercise decisions changes no answer in low dimension and, on the evidence of Example 11.1, none at five dimensions either; it becomes a candidate only where a polynomial basis genuinely runs out, which is higher than the textbook examples suggest. *Validation burden: moderate*—the bound is one-sided, which helps.

Deep hedging changes the answer. It produces a strategy where classical theory produces none, improves expected shortfall by 21% over a tuned band at almost the same cost, and requires the practitioner to state a risk objective that delta hedging allowed them to leave implicit. *Validation burden: high*, because there is nothing to check the answer against.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

The pattern is that the methods with the lowest validation burden are the ones that change the least, which is not a coincidence: a method that only makes a known calculation faster can be checked against the calculation.

15 WHO USES THESE METHODS, AND WHY

Volatility desks use surrogates for calibration, and the motive is the one in Example 2.1: a model too slow to calibrate is a model that cannot be used, whatever its merits. The adoption of rough volatility followed its surrogate rather than its evidence.

XVA desks use differential learning and adjoint differentiation together, because Chapter 9's adjustments are portfolio quantities requiring thousands of revaluations, and Chapter 32 showed the risk run rather than the price to be the binding constraint.

Exotic desks use deep hedging where frictions dominate: illiquid underlyings, large positions relative to the market, and the barrier products of Chapters 34 and 35 whose hedges demand the most trading exactly when trading is most expensive. The reverse knock-out of Example 34.6, with a delta that is negative everywhere and a discontinuity at the barrier, is the natural application—and it is also the case where the validation burden is highest.

Vendors and infrastructure providers supply surrogate libraries, which is how these methods reach desks that have not built them. That distribution channel deserves the scrutiny Section 3 implies: a surrogate arriving as a library is a surrogate whose training box the user did not choose and may not be able to see.

No one is using these methods to discover new derivatives, and in its first decade the field has made existing methods affordable and stated one problem that was previously left unstated. That is a real contribution and it is smaller than the vocabulary suggests.

16 WHERE THIS CAME FROM

Hutchinson, Lo and Poggio (1994) is the pioneering paper and is worth reading for how it framed the problem: a network learns the pricing function from data, and the interesting question is what it has learned when it succeeds.

The modern line differs in ambition. Buehler and co-authors (2019) do not learn a price but a hedging policy, optimizing the terminal distribution directly—which is what deep hedging means and why it can accommodate costs and constraints a formula cannot. Huge and Savine (2020) train on differentials as well as values, which is what makes the Greeks reliable rather than merely available. Horvath and co-authors (2021) apply the surrogate idea to calibration, which is the application this note argues is the most defensible, because there the network is interpolating a function that is known and merely expensive.

SUMMARY

Three activities travel under one name, and one test separates them: does the method change an answer, or only the time taken to get one? Approximating a known pricing function changes no answer, and neither does estimating a conditional expectation

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

from paths, which is Section 20.11's regression with a different basis; only choosing a hedging strategy under frictions states a problem the classical theory cannot. The answer also predicts how much validation each needs.

A surrogate buys speed. A small network fits the Heston implied-volatility map closely and evaluates it far faster than the pricer, so a calibration that took seconds becomes nearly instantaneous. But the training set is the domain of validity: the same network degrades badly at maturities it never saw, and it returns a plausible number rather than an error. Differential learning reaches the same accuracy from a fraction of the paths.

Deep hedging is the one result that is not a speed-up, because under proportional costs replication is impossible. Against a tuned no-trade band the learned policy spends the same money and lowers expected shortfall materially; against naive daily delta the gain looks larger and the cost saving is spurious. The objective is the specification—at zero cost the policy beats delta on expected shortfall and loses to it on standard deviation—and the simulator is an assumption: on jump paths the advantage shrinks to a fraction of its size, and the learned policy degrades most.

What breaks is governance, not mathematics. Different seeds agree closely on expected shortfall while holding materially different deltas, so the shape reproduces and the position does not. The surface stayed arbitrage-free even where pointwise error was large, because a network is smooth—the same smoothness that let one quietly launder a pricer returning negative call prices.

The methods with the lowest validation burden are the ones that change the least, because a method that only makes a known calculation faster can be checked against that calculation.

Companion notebook `tn39-machine-learning.ipynb`

FURTHER READING

Buehler and co-authors' deep hedging paper is the source of Section 6 and states why the problem is a control problem rather than a pricing one; read it for the formulation rather than for the architectures, which have dated. On surrogates, the literature on neural calibration of rough volatility is the case study worth following, because it is the one instance where the technique demonstrably changed which model a market used. Huge and Savine's differential machine learning paper introduced the idea of Section 5 and is explicit that the derivatives were already being computed. The no-trade-region results the learned policy rediscovers are classical—the Whalley–Wilmott asymptotics are the standard reference and are worth reading *before* the machine learning literature, since they establish what the answer should look like. On validation, the supervisory literature is thin and the useful material is in the model risk management guidance written for classical models, read with attention to which of its assumptions a learned model violates.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

PRACTICE QUESTIONS

1 State the three activities that travel under the name "machine learning in derivatives," identify which of them can produce something classical methods do not contain, and explain what that implies for how much validation each requires.

2 Explain why a surrogate is trained on implied volatilities rather than on prices.

3 Using Example 2.1, state the surrogate's RMSE and the calibration speed-up, and explain in one sentence what the surrogate does and does not change.

4 Explain why points with less than 0.1% of spot in time value were excluded from the training set, and what would have gone wrong had they been kept.

5 Using Example 3.1, state the RMSE inside the training box and at five-year maturities, and explain why this failure is more dangerous than a large visible error.

6 Explain the sense in which a surrogate's region of validity is a virtue relative to a closed-form approximation's, and why the practical difficulty is organizational rather than mathematical.

7 In the snapshot of Section 4, state the worst-case error before and after the pricer was corrected, and explain why the network made the underlying bug harder to detect.

8 Using Example 5.1, state the improvement from derivative labels at 4,096 paths, and restate it as a saving in simulation budget rather than in RMSE.

9 Explain why proportional transaction costs make replication impossible rather than merely expensive.

10 Write down the deep hedging objective (TN39.1) and explain what has replaced the requirement that the strategy replicate the payoff.

11 Explain why the current holding must be an input to a cost-aware hedging policy.

12 Using Example 7.1, state the fees and the expected shortfall of the tuned band and of the learned policy at 50 bp, and state in one sentence where the learned policy's advantage comes from.

13 Using Figure 3, describe two ways in which the learned policy differs from the Black–Scholes delta, and explain why the resemblance to the classical no-trade region is reassuring rather than disappointing.

14 At zero transaction costs the learned policy improves expected shortfall by 10.1% and worsens standard deviation by 13.8%. Explain why this is not a defect, and what it implies about a risk report that measures the wrong quantity.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

15 Explain why delta hedging never required a risk objective to be stated, and what that implies about where the arbitrariness in deep hedging actually comes from.

16 Explain why a neural basis for an American Monte Carlo regression buys nothing on a Bermudan swaption, and use Example 11.1 to say whether a callable note on five underlyings needs one.

17 Using Example 12.1, explain why refining the strike grid should have produced a butterfly violation and did not, and state precisely which no-arbitrage conditions a smooth surrogate is and is not protected against.

18 Explain why the property that keeps the fitted surface arbitrage-free is the same one that made the pricer's bug hard to detect.

19 Using Example 10.1, state the learned policy's advantage over the tuned band under the training dynamics and under jumps, and explain why matching total volatility does not make the two worlds equivalent.

20 Rank the three strategies of Example 10.1 by how much each degrades under jumps, and explain why the ranking is what it is.

21 A desk delivers a deep hedging strategy to its risk committee. State what else must be delivered with it, and why that item is the more important of the two.

22 Using Example 12.2, state the spread in expected shortfall and the spread in the holding at a spot of 105 across three seeds, and explain why the first is reassuring and the second is not.

23 Explain what *is* reproducible across training runs in Example 12.2, and why that makes seed dependence a problem for a hedging policy but not for a pricing surrogate.

24 Explain why "is the model conservative?" is a question with no answer for a learned model, and what should be asked instead.

FURTHER QUESTIONS

25 Example 3.1 shows a surrogate degrading badly outside its training box. Design a bounds check that travels with the network rather than with the calling code, determine what it should do when a single input is marginally outside the box while the others are central, and state how you would set the tolerance.

26 The snapshot in Section 4 found bad labels by sorting the test set by error and inspecting the worst points. Devise a procedure that would have caught the negative prices *before* training, state its cost relative to generating the training set, and explain why the obvious check—that prices exceed intrinsic value—is not sufficient on its own.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

27 Example 7.1 trains one policy per cost level. Formulate a single policy that takes the cost rate as an input, state what you would expect its performance to be relative to the three specialized policies, and explain what this implies about how a desk should retrain when its execution costs change.

28 The learned policy in Figure 3 rediscovers the classical no-trade region. Derive the Whalley–Wilmott band width for the parameters of Example 7.1, compare it with the grid-searched width, and determine whether the learned policy's 19% expected-shortfall advantage survives against the analytic band rather than the grid-searched one.

29 Section 9 argues that the risk measure is part of the specification. Train the same policy under variance and under an exponential utility, compare the three hedges, and state which features of the strategy are common to all three and which are artifacts of the objective.

30 Deep hedging produces a strategy but not a price. Construct an indifference price from the objective in (TN39.1), explain how it depends on the risk measure and on the existing book, and state what this implies for two desks quoting the same exotic.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!